

tunn3l v1s10n

 **Security Warning:** Work in an isolated environment (virtual machine or container). Never open unknown files on your primary machine.

Challenge Description

We found this file. Recover the flag.

Hints

- Weird that it won't display right...
-

Tools Used

- `file` – Determine file type
 - `strings` – Extract printable character sequences
 - `exiftool` – Read metadata
 - `xxd` – View hexadecimal representation of files
 - `hexedit` – Interactive hexadecimal editor
 - **Image viewer** – Display the final image
-

Complete Solution

Step 1: Download and Prepare

Download the file named `tunn3l_v1s10n` from the challenge page.

Step 2: Basic File Inspection

First, verify the file type:

```
file tunn3l_v1s10n
```

Expected output:

```
tunn3l_v1s10n: data
```

The `file` command doesn't recognize the file type – it's corrupted or has a missing/incorrect header.

Step 3: Search for Visible Text

Check if any readable text reveals the flag:

```
strings tunn3l_v1s10n | grep -i "picoCTF"
```

Output: (no results)

No flag yet.

Step 4: Examine Metadata

Use `exiftool` to inspect the file's metadata:

```
exiftool tunn3l_v1s10n
```

Output:

```
ExifTool Version Number      : 13.50
File Name                    : tunn3l_v1s10n
Directory                   : .
File Size                    : 2.9 MB
File Modification Date/Time   : 2026:05:20 12:25:27+02:00
...
File Type                    : BMP
File Type Extension          : bmp
```

MIME Type : image/bmp
BMP Version : Unknown (53434)
Image Width : 1134
Image Height : 306
Planes : 1
Bit Depth : 24
Compression : None
Image Length : 2893400
...

Key finding: The file is identified as a **BMP image** with `Unknown` version. The unusual `BMP Version` value (`53434`) suggests the header is corrupted.

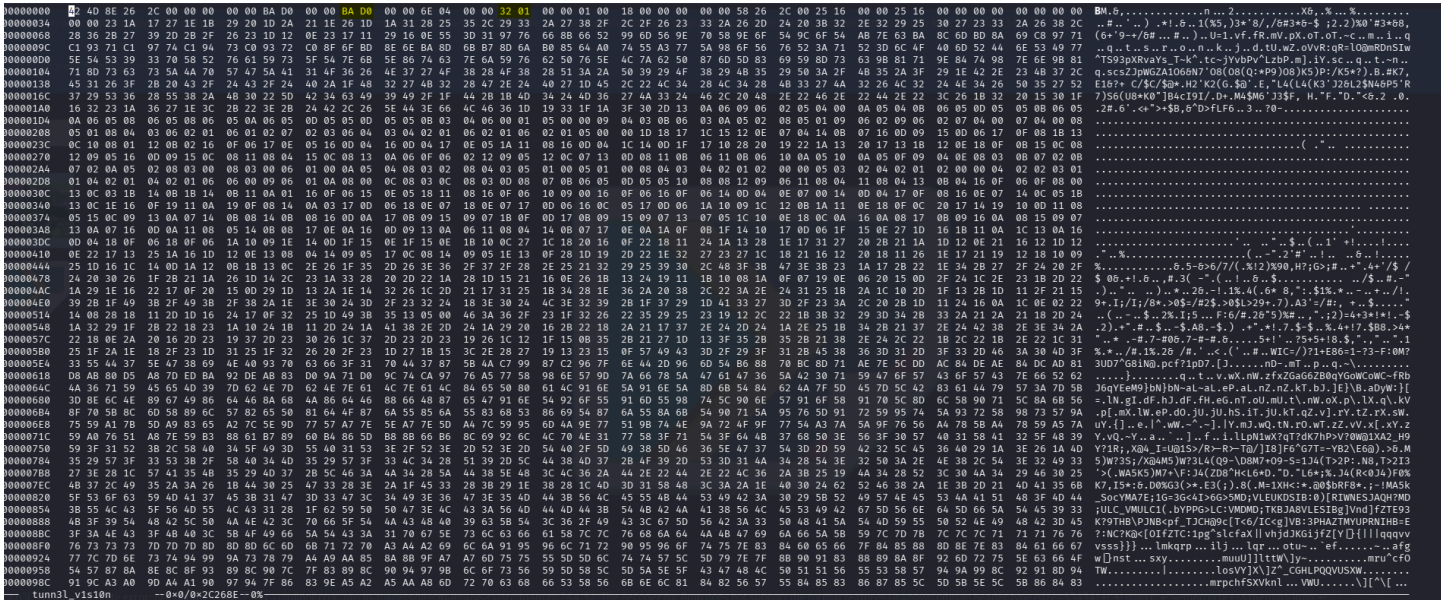
Step 5: Hexdump Analysis

Let’s examine the hexadecimal representation of the file header:

```
xxd tunn3l_v1s10n | head
```

Output:

```
00000000: 424d 8e26 2c00 0000 0000 bad0 0000 bad0  BM.&,.....
00000010: 0000 6e04 0000 3201 0000 0100 1800 0000  ..n...2.....
00000020: 0000 5826 2c00 2516 0000 2516 0000 0000  ..X&,.%...%....
00000030: 0000 0000 0000 231a 1727 1e1b 2920 1d2a  .....#...'..) .*
00000040: 211e 261d 1a31 2825 352c 2933 2a27 382f  !.&..1(%5,)3*'8/
00000050: 2c2f 2623 332a 262d 2420 3b32 2e32 2925  ,/&#3*&-$ ;2.2)%
00000060: 3027 2333 2a26 382c 2836 2b27 392d 2b2f  0'#3*&8,(6+'9-+/
00000070: 2623 1d12 0e23 1711 2916 0e55 3d31 9776  &#...#..) ..U=1.v
00000080: 668b 6652 996d 569e 7058 9e6f 549c 6f54  f.fR.mV.pX.oT.oT
00000090: ab7e 63ba 8c6d bd8a 69c8 9771 c193 71c1  .~c..m..i..q..q.
```



Step 6: Understand BMP File Structure

A valid BMP file has a specific header structure:

Offset	Size	Field	Value in our file	Should be
0x00	2 bytes	Signature	42 4D (BM)	✔ Correct
0x0A	4 bytes	Offset to pixel data	bad0 0000	✗ Should point after header
0x12	4 bytes	Width	3201 0000 (306)	✔ 306 is plausible
0x16	4 bytes	Height	0000 0000	✗ Should be positive

The main issues are:

- The offset to pixel data (0x0A) contains bad0 – this is not a standard value
- The height (0x16) is zero, which is invalid

Step 7: Fix the Header

We need to correct the BMP header. Let's check a reference BMP header:

```
Offset 0x00-0x01: 42 4D (BM)
Offset 0x02-0x05: File size
Offset 0x0A-0x0D: Offset to pixel data (usually 54 = 0x36 for 24-bit BMP)
```

In our file, the pixel data offset appears to be corrupted. Let's change it to `36 00 00 00` (54 decimal).

Also, the height field at offset `0x16` should be positive. Looking at the image dimensions in `exiftool`, we saw `Image Length: 2893400` and width `1134`. Let's calculate the correct height:

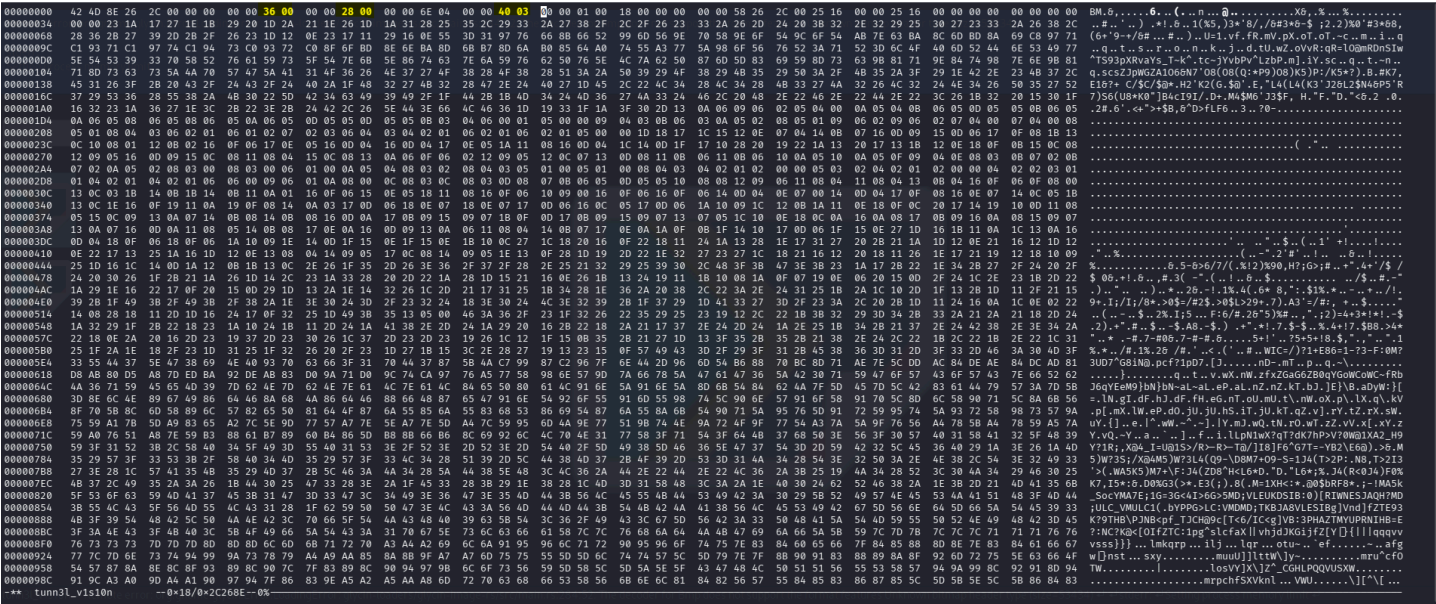
- File size: ~2.9 MB
- Width: 1134 pixels
- Bit depth: 24 bits = 3 bytes per pixel
- Expected height = (File size - header size) / (width × 3)

Using `hexedit` to modify the file:

```
hexedit tunn3l_v1s10n
```

Changes to make:

Offset	Original	Change to	Reason
0x0A	bad0 0000	36 00 00 00	Standard BMP pixel data offset (54 bytes)
0x16	3201 0000	40 03 00 00	Height = 832 (0x0340) to match image data



After modifications:

```
xxd tunn3l_v1s10n | head
```

Output:

```
00000000: 424d 8e26 2c00 0000 0000 3600 0000 2800 BM.&,.....6...(.
00000010: 0000 6e04 0000 4003 0000 0100 1800 0000 ..n...@.....
00000020: 0000 5826 2c00 2516 0000 2516 0000 0000 ..X&,.%...%.....
00000030: 0000 0000 0000 231a 1727 1e1b 2920 1d2a .....#..'..) .*
00000040: 211e 261d 1a31 2825 352c 2933 2a27 382f !.&..1(%5,)3*'8/
00000050: 2c2f 2623 332a 262d 2420 3b32 2e32 2925 ,/&#3*&-$ ;2.2)%
00000060: 3027 2333 2a26 382c 2836 2b27 392d 2b2f 0'#3*&8,(6+'9-+/
00000070: 2623 1d12 0e23 1711 2916 0e55 3d31 9776 &#...#...)..U=1.v
00000080: 668b 6652 996d 569e 7058 9e6f 549c 6f54 f.fR.mV.pX.oT.oT
00000090: ab7e 63ba 8c6d bd8a 69c8 9771 c193 71c1 .~c..m..i..q..q.
```

Step 8: Verify the Repaired File

Check if the file is now recognized:

```
file tunn3l_v1s10n
```

Output:

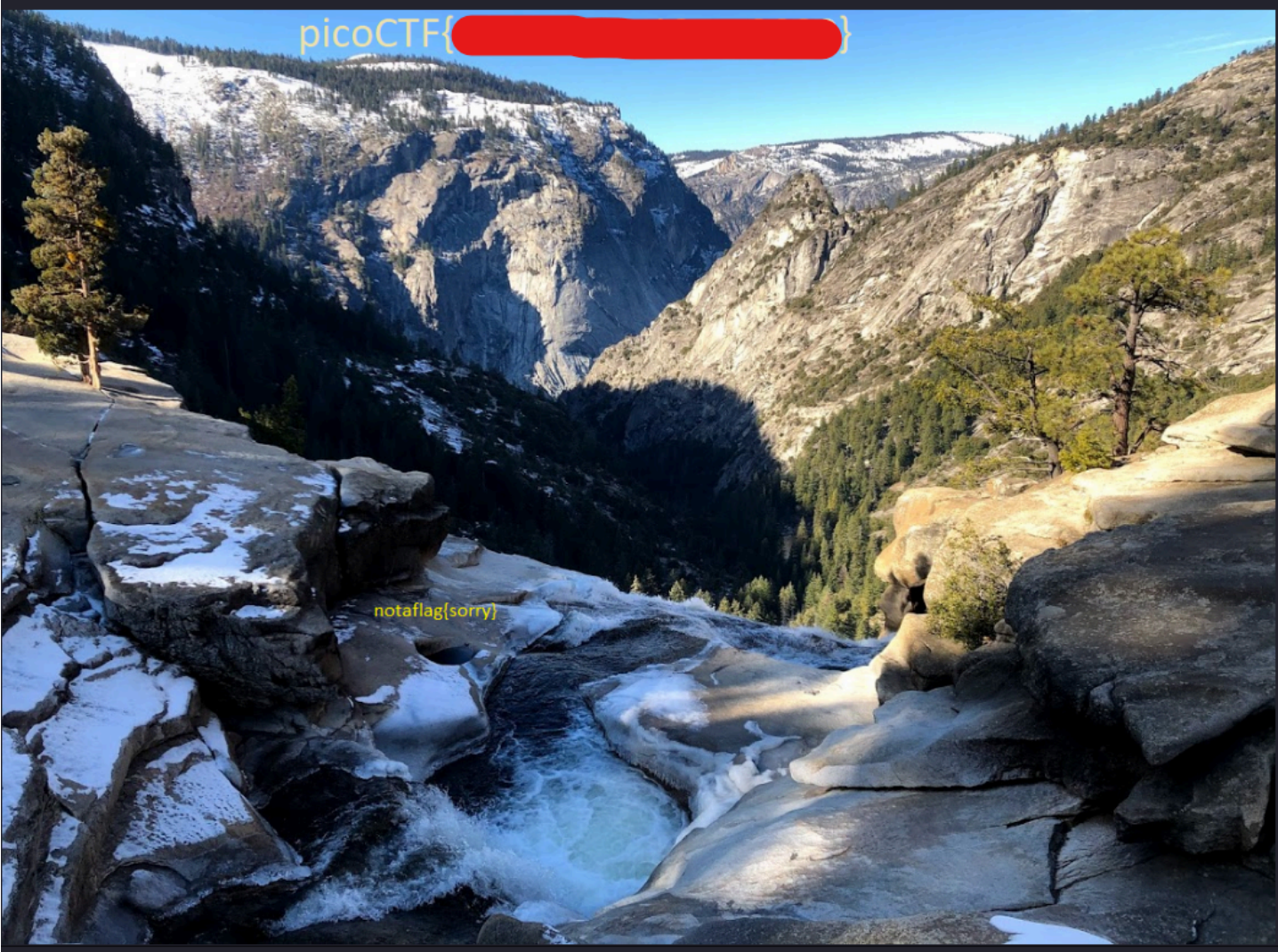
```
tunn3l_v1s10n: PC bitmap, Windows 3.x format, 1134 x 832 x 24, image size 2893400,
resolution 5669 x 5669 px/m, cbSize 2893454, bits offset 54
```

Success! The file is now a valid BMP image with dimensions 1134×832 pixels.

Step 9: Open the Image

Rename the file with the correct extension and open it:

```
cp tunn3l_v1s10n flag.bmp
xdg-open flag.bmp    # Linux
open flag.bmp        # macOS
start flag.bmp       # Windows
```

The flag is visible directly in the image!

🧠 Key Concepts

Concept	Description
Magic Bytes (File Signatures)	Unique byte sequences at the start of a file that identify its format. BMP files start with 42 4D (BM).
BMP Header Structure	Contains signature, file size, pixel data offset, width, height, bit depth, and compression type.
Pixel Data Offset	The location in the file where actual image pixels begin (usually 54 bytes for 24-bit BMP).
Image Dimensions	Width and height fields must match the actual pixel data size.

Concept	Description
xxd	Tool to view and manipulate hexadecimal representations of files.
hexedit	Interactive hexadecimal editor for direct file modification.
Corruption Repair	Fixing corrupted file headers by comparing with known file format specifications.

Pro Tips

1. **exiftool often reveals the true file type** – Even when the header is corrupted, metadata may still be readable.
2. **BMP files start with BM** – This is the magic signature for Windows Bitmap files.
3. **The pixel data offset is usually 54** – For 24-bit uncompressed BMP, the header is 54 bytes (`0x36`).
4. **Calculate dimensions from file size** – If width is known, height = (file size - offset) / (width × bytes per pixel).
5. **Use hexedit for precision** – It allows you to see and modify bytes in real-time.
6. **Compare with a known good file** – Create a dummy BMP and compare its header with the corrupted one.